

Tugas Pertemuan 11 – Peta Kota Struktur Data Graph

1. Source Code

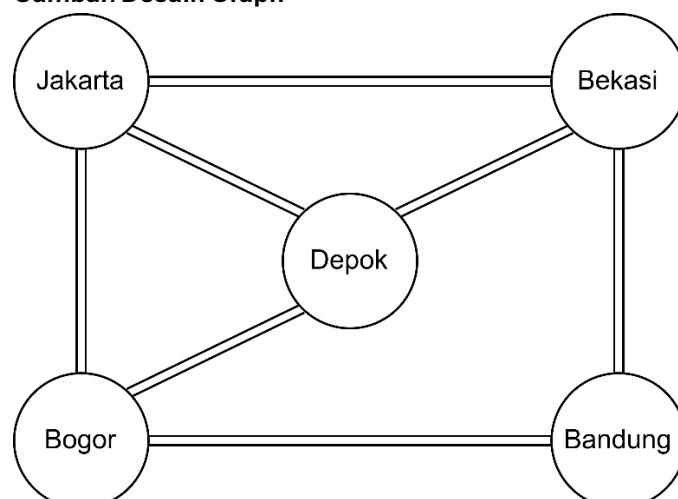
Berikut adalah *source code* dari Program yang Saya buat:

[Click Me!](#)

2. Screenshot Hasil Program

```
1  # Euwlipin Natanoel Saudale / J0403251016 / TPL A1
2
3  # --- PROGRAM GRAPH PETA KOTA ---
4
5  # 1. Definisi Node
6  nodes = ["Jakarta", "Bogor", "Bandung", "Depok", "Bekasi"]
7
8  # 2. Representasi Adjacency List (Pakai Dictionary)
9  # Menunjukkan tiap kota nyambung ke mana saja
10 adj_list = {
11     "Jakarta": ["Bogor", "Bekasi", "Depok"],
12     "Bogor": ["Jakarta", "Bandung", "Depok"],
13     "Bandung": ["Bogor", "Bekasi"],
14     "Depok": ["Jakarta", "Bogor", "Bekasi"],
15     "Bekasi": ["Jakarta", "Bandung", "Depok"]
16 }
17
18 # 3. Representasi Adjacency Matrix
19 # 0 = Tidak ada jalan, 1 = Ada jalan
20 # Urutan: Jakarta, Bogor, Bandung, Depok, Bekasi
21 adj_matrix = [
22     [0, 1, 0, 1, 1], # Jakarta
23     [1, 0, 1, 1, 0], # Bogor
24     [0, 1, 0, 0, 1], # Bandung
25     [1, 1, 0, 0, 1], # Depok
26     [1, 0, 1, 1, 0] # Bekasi
27 ]
28
29 # --- OUTPUT PROGRAM ---
30 print("=== DAFTAR NODE (KOTA) ===")
31 for i, node in enumerate(nodes):
32     print(f"{i}. {node}")
33
34 print("\n=== ADJACENCY LIST (HUBUNGAN LANGSUNG) ===")
35 for kota, tetangga in adj_list.items():
36     print(f"{kota} terhubung ke: {' '.join(tetangga)}")
37
38 print("\n=== ADJACENCY MATRIX ===")
39 print("    JK BG BD DP BK") # Header singkat
40 for i, row in enumerate(adj_matrix):
41     print(f"{nodes[i][:2]} {row}")
```

3. Gambar/Desain Graph



4. Penjelasan Node dan Edge

Di sini intinya cuma dua:

- Node (Vertex): Ini adalah "objek" utamanya. Kalau di peta, ya nama kotanya.
- Edge: Ini adalah "hubungannya". Di sini, edge adalah jalan yang menghubungkan dua kota.

Daftar Node:

1. Jakarta
2. Bogor
3. Bandung
4. Depok
5. Bekasi

Daftar Edge (Hubungan antar kota):

1. Jakarta <---> Bogor
2. Jakarta <---> Bekasi
3. Jakarta <---> Depok
4. Bogor <---> Bandung
5. Bogor <---> Depok
6. Bekasi <---> Bandung
7. Bekasi <---> Depok

5. Analisis

Berikut adalah analisis singkat dari tugas studi kasus diatas:

1. Jenis Graph: Graph ini adalah Undirected Graph (tidak berarah). Alasannya, karena jalan dari Jakarta ke Bogor biasanya bisa dilewati bolak-balik. Tidak ada tanda panah satu arah saja.
2. Alasan Memilih Edge: Edge dipilih berdasarkan kedekatan geografis. Misalnya, Jakarta ke Bandung tidak dibuat langsung (harus lewat Bogor atau Bekasi dulu) untuk mencerminkan rute perjalanan nyata di Jawa Barat.
3. Representasi Mana yang Lebih Cocok?
 - Untuk kasus peta ini, Adjacency List lebih cocok. Kenapa? Karena lebih enak dibaca manusia (langsung kelihatan nama kotanya) dan lebih hemat memori kalau kotanya ada ribuan tapi jalannya cuma dikit.
 - Adjacency Matrix lebih cocok kalau kita mau proses cepat pakai matematika (misal buat nyari rute terpendek pakai algoritma tertentu), tapi boros tempat karena banyak angka 0-nya.
4. Sparse atau Dense? Graph ini termasuk Sparse Graph (jarang). Jumlah edge kita (7) masih jauh dari jumlah maksimal edge yang mungkin ada ($5 \times 4 / 2 = 10$). Masih banyak kota yang tidak terhubung langsung satu sama lain.

6. Kesimpulan

Graph sangat membantu kita memvisualisasikan data yang saling berhubungan seperti rute peta. Dengan mengubah objek nyata menjadi Node dan Edge, kita bisa menyelesaikannya secara sistematis menggunakan struktur data di Python, baik lewat list (yang ringkas) atau matrix (yang matematis).